

Code-based PKE based on Binary Goppa Code

DSC 4110/6111: Modern Cryptography and AI/ML Security

Dr. Laltu Sardar

School of Data Science,
Indian Institute of Science Education and Research Thiruvananthapuram (IISER TVM)



September 11 & 12, 2026



Table of Content

1 Goppa-Code

2 Code-Based Cryptography

The next part explains a specific construction of a code-based public-key encryption scheme.
This part will be excluded from the exam.

Binary Goppa-Code

An Error-correcting code for Code-based PKE

Why Goppa Codes?

Question

We want a code with two properties:

- 1 Efficient decoding for the legitimate receiver.
- 2 A decoding problem that is hard for an attacker without secret information.

Why Goppa Codes?

Question

We want a code with two properties:

- 1 Efficient decoding for the legitimate receiver.
- 2 A decoding problem that is hard for an attacker without secret information.

A particularly important family for code-based cryptography is the **binary Goppa code**.

Goppa Code

Algebraic structure + Efficient decoding

Goppa codes form the mathematical foundation of the original **McEliece cryptosystem**.

Question

What special structure allows a receiver to decode efficiently?

Idea Behind a Goppa Code

Let

$$\mathbb{F}_{2^m}$$

be a finite field.

Choose:

- a set of distinct support elements

$$L = (\alpha_1, \alpha_2, \dots, \alpha_n), \quad \alpha_i \in \mathbb{F}_{2^m},$$

- a polynomial

$$g(x) \in \mathbb{F}_{2^m}[x]$$

of degree t .

The polynomial $g(x)$ is called the **Goppa polynomial**.

We require

$$g(\alpha_i) \neq 0 \quad \text{for all } i.$$

Question

Why should a polynomial and a set of field elements define a code?

Idea Behind a Goppa Code

Let

$$\mathbb{F}_{2^m}$$

be a finite field.

Choose:

- a set of distinct support elements

$$L = (\alpha_1, \alpha_2, \dots, \alpha_n), \quad \alpha_i \in \mathbb{F}_{2^m},$$

- a polynomial

$$g(x) \in \mathbb{F}_{2^m}[x]$$

of degree t .

The polynomial $g(x)$ is called the **Goppa polynomial**.

We require

$$g(\alpha_i) \neq 0 \quad \text{for all } i.$$

Question

Why should a polynomial and a set of field elements define a code?

The answer comes from imposing an algebraic condition on the vectors $x = (x_1, \dots, x_n) \in \mathbb{F}_2^n$.

Construction of a Binary Goppa Code

Given $g(x)$ and

$$L = (\alpha_1, \dots, \alpha_n),$$

define the binary Goppa code

$$\Gamma(L, g)$$

as the set of vectors

$$c = (c_1, \dots, c_n) \in \mathbb{F}_2^n$$

satisfying

$$\sum_{i=1}^n \frac{c_i}{x - \alpha_i} \equiv 0 \pmod{g(x)}.$$

Equivalently,

$$\Gamma(L, g) = \left\{ c \in \mathbb{F}_2^n : \sum_{i=1}^n \frac{c_i}{x - \alpha_i} \equiv 0 \pmod{g(x)} \right\}.$$

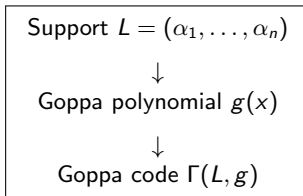
Important parameters

For a binary Goppa code with $\deg(g) = t$, $k \geq n - mt$.

The code has designed minimum distance at least $d \geq 2t + 1$.

A Small Structural View

The construction uses two ingredients:



where

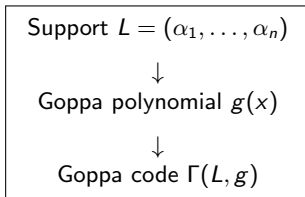
$$\alpha_i \in \mathbb{F}_{2^m}, \quad \deg(g) = t, \quad g(\alpha_i) \neq 0.$$

Question

What determines how many errors the code can correct?

A Small Structural View

The construction uses two ingredients:



where

$$\alpha_i \in \mathbb{F}_{2^m}, \quad \deg(g) = t, \quad g(\alpha_i) \neq 0.$$

Question

What determines how many errors the code can correct?

The degree t of the Goppa polynomial determines the designed error-correction capability:

Up to t errors can be corrected.

This is one of the important reasons Goppa codes are attractive for cryptography.

How Does Encoding Work?

A Goppa code is a **linear code**.

Therefore, once we obtain a generator matrix

$$G \in \mathbb{F}_2^{k \times n},$$

a message

$$m \in \mathbb{F}_2^k$$

is encoded as

$$\boxed{c = mG}.$$

Thus,

$$\mathbb{F}_2^k \xrightarrow{G} \Gamma(L, g) \subseteq \mathbb{F}_2^n.$$

Question

Does the Goppa polynomial itself directly perform the encoding?

How Does Encoding Work?

A Goppa code is a **linear code**.

Therefore, once we obtain a generator matrix

$$G \in \mathbb{F}_2^{k \times n},$$

a message

$$m \in \mathbb{F}_2^k$$

is encoded as

$$c = mG.$$

Thus,

$$\mathbb{F}_2^k \xrightarrow{G} \Gamma(L, g) \subseteq \mathbb{F}_2^n.$$

Question

Does the Goppa polynomial itself directly perform the encoding?

Not usually.

The Goppa polynomial and support define the code. We can then derive a generator matrix G for that code.

Code structure $\rightarrow G \rightarrow$ Encoding

Generator Matrix of a Goppa Code

Let

$$H \in \mathbb{F}_2^{(n-k) \times n}$$

be a parity-check matrix for the Goppa code.

Then

$$\Gamma(L, g) = \{c \in \mathbb{F}_2^n : Hc^T = 0\}.$$

A generator matrix G satisfies

$$HG^T = 0.$$

The rows of G form a basis of the Goppa code.

Therefore every codeword has the form

$$c = mG.$$

Question

If we know G , what information does it reveal?

It completely specifies the linear code as a vector space.

This observation will become crucial when we construct a public-key encryption scheme.

Parity-Check Matrix of a Goppa Code

A parity-check matrix can be constructed from the support and Goppa polynomial.
A common representation over $\mathbb{F}_{2^m}$ is

$$H = \begin{bmatrix} 1 & 1 & \cdots & 1 \\ \frac{1}{g(\alpha_1)} & \frac{1}{g(\alpha_2)} & \cdots & \frac{1}{g(\alpha_n)} \\ \alpha_1 & \alpha_2 & \cdots & \alpha_n \\ \frac{\alpha_1}{g(\alpha_1)} & \frac{\alpha_2}{g(\alpha_2)} & \cdots & \frac{\alpha_n}{g(\alpha_n)} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\alpha_1^{t-1}}{g(\alpha_1)} & \frac{\alpha_2^{t-1}}{g(\alpha_2)} & \cdots & \frac{\alpha_n^{t-1}}{g(\alpha_n)} \end{bmatrix}.$$

Each entry lies in $\mathbb{F}_{2^m}$.

Expanding each field element into its m binary coordinates gives a binary parity-check matrix.

Question

What does $Hc^T = 0$ tell us?

c satisfies all parity constraints of the Goppa code.

Decoding a Goppa Code

Suppose a codeword c is transmitted through a noisy channel.
The receiver obtains

$$r = c + e,$$

where

$$\text{wt}(e) \leq t.$$

Question

What does the receiver need to recover?

The receiver needs to determine the error vector e .
Once e is known,

$$c = r - e.$$

For a Goppa code, the receiver computes the **syndrome**

$$s = Hr^T.$$

Since

Goppa Decoding: The Algebraic Idea

The receiver has

$$s = He^T.$$

The goal is to determine the error positions and values.

A Goppa decoder uses algebraic algorithms to recover the error vector.

Conceptually:

$$r \longrightarrow s \longrightarrow \text{error-locator polynomial} \longrightarrow \text{error positions} \longrightarrow e \longrightarrow c.$$

For binary Goppa codes, efficient algebraic decoding algorithms can correct up to t errors.

Question

Why can the legitimate receiver decode efficiently?

Goppa Decoding: The Algebraic Idea

The receiver has

$$s = He^T.$$

The goal is to determine the error positions and values.

A Goppa decoder uses algebraic algorithms to recover the error vector.

Conceptually:

$$r \longrightarrow s \longrightarrow \text{error-locator polynomial} \longrightarrow \text{error positions} \longrightarrow e \longrightarrow c.$$

For binary Goppa codes, efficient algebraic decoding algorithms can correct up to t errors.

Question

Why can the legitimate receiver decode efficiently?

Because the receiver knows the special algebraic structure defined by

$$\boxed{(L, g)}.$$

This structure enables the efficient Goppa decoding algorithm.

McEliece Encryption Scheme

A Code-Based PKE from Goppa Code

The Key Cryptographic Property

Now consider the two situations.

Legitimate Receiver

Knows

$$L, \quad g(x)$$

and therefore knows the special structure of the Goppa code.

Efficient decoding

Attacker

Sees a public representation that should hide the special Goppa structure.

Decoding is hard

Answer

The property providing easy decoding with secret information is the **hidden algebraic structure of the Goppa code**, specifically the knowledge of the support L and Goppa polynomial $g(x)$ that enables the efficient Goppa decoding algorithm.

Why Is This Useful for Public-Key Cryptography?

We now have an interesting situation:

Secret knowledge of (L, g)



Efficient decoding

while we want the public representation to hide this structure:

Public representation



Hard decoding problem

Question

Can we transform the generator matrix of the Goppa code so that:

- Bob can still use his secret Goppa structure,
- Eve sees only a disguised generator matrix?

Goppa Code $\rightarrow$ Hidden Structure $\rightarrow$ McEliece PKE

From Goppa Code to McEliece PKE

Assume we have constructed a binary Goppa code

$$\Gamma(L, g) \subseteq \mathbb{F}_2^n$$

with parameters $[n, k, t]$ and generator matrix

$$G \in \mathbb{F}_2^{k \times n}.$$

Recall that G maps a k -bit message to an n -bit codeword:

$$c = mG.$$

Problem: If G is public, anyone can exploit the Goppa structure and efficiently decode corrupted codewords.

Idea: Hide the structure of G using two secret transformations:

$$G_{\text{pub}} = SGP$$

where S is a random invertible $k \times k$ binary matrix and P is a random $n \times n$ permutation matrix.

Secret key: (S, G, P) Public key: G_{pub}

McEliece PKE: Encryption

The sender knows only the public key

$$G_{\text{pub}} = SGP.$$

For a message $m \in \mathbb{F}_2^k$, first compute the public-code codeword:

$$c = mG_{\text{pub}}.$$

Then choose a random error vector $e \in \mathbb{F}_2^n$ satisfying

$$\text{wt}(e) \leq t.$$

The ciphertext is

$$y = c + e = mG_{\text{pub}} + e.$$

$$m \xrightarrow{G_{\text{pub}}} c \xrightarrow{+e} y$$

McEliece PKE: Decryption

The receiver knows (S, G, P) and receives $y = mSGP + e$.

Step 1: Undo the permutation

$$\boxed{yP^{-1} = mSG + e'}, \quad e' = eP^{-1}.$$

Since P is a permutation,

$$\text{wt}(e') = \text{wt}(e) \leq t.$$

Step 2: Goppa decoding Using the secret Goppa structure (L, g) :

$$\boxed{yP^{-1} \xrightarrow{\text{Goppa decoder}} mSG}$$

Step 3: Recover the message From mSG , recover mS and compute

$$\boxed{m = (mS)S^{-1}}.$$

$$\boxed{y \xrightarrow{P^{-1}} mSG + e' \xrightarrow{\text{Goppa Decode}} mSG \xrightarrow{S^{-1}} m}$$

Security Assumptions of McEliece PKE

The attacker knows

$$G_{\text{pub}} = SGP$$

and receives

$$y = mG_{\text{pub}} + e.$$

To decrypt, the attacker would need to recover m from y .

There are two essential obstacles:

- 1 **Remove the error:** Given $y = mSGP + e$, recover the error e or the underlying codeword $mSGP$.
- 2 **Recover the hidden structure:** Even after obtaining the codeword, the attacker needs to undo the hidden transformations S and P to recover m .

The security relies on the assumption that, given only

$$G_{\text{pub}} = SGP,$$

it is computationally infeasible to recover the hidden Goppa structure or efficiently decode the public code.

Recover secret structure / efficiently decode G_{pub} is computationally hard

Textbook:

- 1 Daniel J. Bernstein, Johannes Buchmann, and Erik Dahmen. *Post-Quantum Cryptography*. Springer, 2009. ISBN: 9783540887010.

Acknowledgment:

- Some figures in this work were generated with the assistance of ChatGPT (OpenAI). It is used to decorate contents as well.



Dr. Laltu Sardar, Assistant Professor, IISER Thiruvananthapuram
laltu.sardar@iisertvm.ac.in, laltu.sardar.crypto@gmail.com
Course webpage: https://laltu-sardar.github.io/courses/26_crypto.html.